

Apple Inc.

Think different.



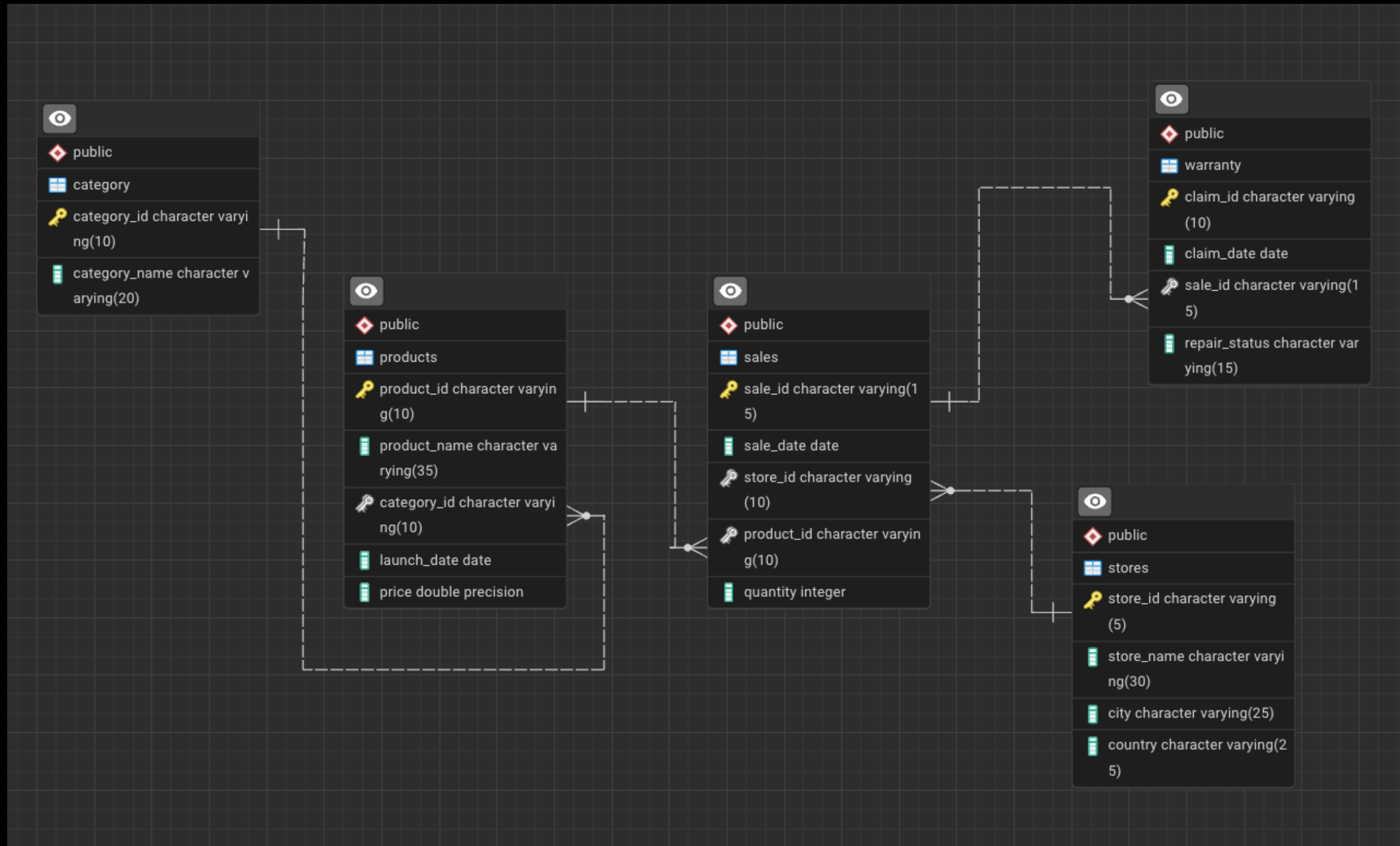
Apple Retail Sales Analysis SQL Project Report

Prepared by: Saiful Islam Rupom

Overview

This project showcases advanced SQL techniques by analyzing 500K+ global retail sales records of Apple Inc. to solve 20+ real-world business problems, including product performance, warranty trends, and sales growth.

Entity Relationship Diagram (ERD)



Key Business Questions

1. Identify the number of stores available in each country.
2. Determine the total units sold by each store.
3. Determine the number of sales transactions that occurred in December 2023.
4. Identify the number of stores with no recorded warranty claims.
5. Determine the percentage of warranty claims categorized as 'Warranty Void'.
6. Determine the store with the highest total units sold in 2024.
7. Calculate the total quantity of each product sold between 2020 and 2024.
8. Calculate the average price of products within each category.
9. What is the total number of warranty claims filed in 2020, 2021, and 2024 combined?
10. For each store, identify the best-selling day of the week based on highest quantity of units sold.

Key Business Questions

11. Identify the least selling product in each country for each year based on total units sold.
12. Determine the number of warranty claims filed within 180 days of the corresponding product sale.
13. For products launched within the last two years, calculate the total units sold and the number of those units with associated warranty claims.
14. Identify the months within the past three years during which sales in the USA exceeded 2500 units.
15. Identify the product category along with the number of warranty claims filed in the last two years.

Key Business Questions

16. Calculate the percentage of chance of receiving warranty claims after each purchase for each country.
17. Analyze the year-by-year growth ratio for each store.
18. Calculate the relationship between product price and warranty claims for products sold in the last three years, segmented by price range.
19. Identify the store with the highest percentage of 'Paid Repaired' claims relative to the total number of claims filed.
20. Calculate the 'monthly running total' and the 'rolling average of the last 3 months' of sales for each store.
21. Analyze product sales trends over time, segmented into key periods: from launch to 6 months, 6-12 months, 12-18 months, and beyond 18 months.

Problem statement 1: Identify the number of stores available in each country.

Solution:

```
SELECT
    country,
    COUNT(*) AS total_stores
FROM stores
GROUP BY country
ORDER BY total_stores DESC;
```

	country character varying (25) 🔒	total_stores bigint 🔒	
1	UK	10	
2	USA	10	
3	Canada	8	
4	China	4	
5	Australia	3	
6	Brazil	2	
7	Japan	2	
8	Spain	2	
9	Turkey	2	
10	India	2	
11	France	2	
Total rows: 35		Query complete 00:00:00.071	

Problem statement 2: Determine the total units sold by each store.

Solution:

```
SELECT
    st.store_id,
    st.store_name,
    SUM(sa.quantity) AS total_units_sold
FROM sales sa
JOIN stores st
    ON sa.store_id = st.store_id
GROUP BY st.store_id
ORDER BY total_units_sold DESC;
```

	store_id [PK] character varying (5)	store_name character varying (30)	total_units_sold bigint
1	ST-52	Apple Mumbai	24590
2	ST-53	Apple New Delhi	24501
3	ST-51	Apple Seoul	24469
4	ST-14	Apple Pacific Centre	24349
5	ST-54	Apple Ankara	24286
6	ST-55	Apple Istanbul	24105
7	ST-21	Apple Orchard Road	23094
8	ST-23	Apple Covent Garden	23036
9	ST-22	Apple Regent Street	23030
10	ST-24	Apple Bluewater	22906
11	ST-25	Apple Munich	22888
Total rows: 64		Query complete 00:00:00.197	

Problem statement 3: Determine the number of sales transactions that occurred in December 2023.

Solution:

```
SELECT COUNT(*) AS dec_2023_total_sales
FROM sales
WHERE sale_date >= '2023-12-01' AND sale_date <= '2023-12-31';
```

	dec_2023_total_sales 
1	23658

Problem statement 4: Identify the number of stores with no recorded warranty claims.

Solution:

```
SELECT COUNT(*)  
FROM stores st  
WHERE st.store_id NOT IN(  
    SELECT DISTINCT sa.store_id FROM sales sa  
    RIGHT JOIN warranty w  
    ON sa.sale_id = w.sale_id  
);|
```

	count bigint 
1	58

Problem statement 5: Determine the percentage of warranty claims categorized as 'Warranty Void'.

Solution:

```
SELECT  
  ROUND((((SELECT COUNT(*) FROM warranty WHERE repair_status = 'Warranty Void')* 100.0)  
  /(SELECT COUNT(*) FROM warranty)),2)  
  AS warranty_void_percentage;
```

	warranty_void_percentage 
1	24.26

Problem statement 6: Determine the store with the highest total units sold in 2024.

Solution:

```
SELECT
    st.store_id,
    st.store_name,
    SUM(sa.quantity) AS units_sold
FROM sales sa
JOIN stores st
    ON sa.store_id = st.store_id
WHERE sa.sale_date >= '2024-01-01' AND sa.sale_date <= '2024-12-31' -- Year 2024
GROUP BY st.store_id, st.store_name
ORDER BY units_sold DESC
LIMIT 1;
```

	store_id [PK] character varying (5) 	store_name character varying (30) 	units_sold bigint 
1	ST-15	Apple Chinook Centre	209

Problem statement 7: Calculate the total quantity of each product sold between 2020 and 2024.

Solution:

```
SELECT
  p.product_id,
  p.product_name,
  SUM(sa.quantity) AS total_units_sold
FROM sales sa
JOIN products p
  ON sa.product_id = p.product_id
WHERE sa.sale_date >= '2020-01-01' AND sa.sale_date <= '2024-12-31' -- Year(2020-2024)
GROUP BY p.product_id, p.product_name
ORDER BY total_units_sold DESC;
```

	product_id [PK] character varying (10)	product_name character varying (35)	total_units_sold bigint
1	P-40	AirTag	92726
2	P-49	MacBook Pro (M1 Max, 16-i...	33682
3	P-48	MacBook Pro (M1 Pro, 14-i...	33581
4	P-55	iPhone 14	31944
5	P-54	iPhone 14 Pro	31728
6	P-58	AirPods Pro (2nd Gen)	20807
7	P-60	iPad Pro (M2, 11-inch)	20747
8	P-59	iPad (10th Gen)	20687
9	P-16	iPhone XR	17569
10	P-15	iPhone XS Max	17218

Problem statement 8: Calculate the average price of products within each category.

Solution:

```
SELECT
    c.category_id,
    c.category_name,
    AVG(p.price) AS average_price
FROM category c
JOIN products p
    ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name
ORDER BY average_price DESC;
```

	category_id [PK] character varying (10) 	category_name character varying (20) 	average_price double precision 
1	CAT-1	Laptop	1511.5
2	CAT-7	Desktop	1199
3	CAT-4	Smartphone	889.4347826086956
4	CAT-3	Tablet	656.5
5	CAT-5	Wearable	362.3333333333333
6	CAT-2	Audio	259
7	CAT-6	Streaming Device	179
8	CAT-8	Subscription Service	149.5
9	CAT-9	Smart Speaker	99
10	CAT-10	Accessory	29
Total rows: 10		Query complete 00:00:00.118	

Problem statement 9: What is the total number of warranty claims filed in 2020, 2021, and 2024 combined?

Solution:

```
SELECT
    COUNT(*) AS number_of_warranty_claim
FROM warranty
WHERE EXTRACT(YEAR FROM claim_date) IN (2020, 2021, 2024);
```

	number_of_warranty_claim 
1	16479

Problem statement 10: For each store, identify the best-selling day of the week based on highest quantity of units sold.

Solution:

```
SELECT
  t.store_id,
  st.store_name,
  t.day_name,
  t.total_units_sold
FROM(
  SELECT
    store_id,
    TO_CHAR(sale_date, 'Day') as day_name,
    SUM(quantity) AS total_units_sold,
    DENSE_RANK() OVER(PARTITION BY store_id ORDER BY SUM(quantity) DESC) AS rank
  FROM sales
  GROUP BY store_id, day_name
) t
JOIN stores st
  ON t.store_id = st.store_id
WHERE rank = 1;
```

	store_id character varying (10) 🔒	store_name character varying (30) 🔒	day_name 🔒	total_units_sold 🔒 bigint
1	ST-1	Apple Fifth Avenue	Friday	1122
2	ST-10	Apple Tysons Corner	Sunday	1078
3	ST-11	Apple Eaton Centre	Saturday	1106
4	ST-12	Apple Yorkdale	Sunday	1074
5	ST-13	Apple Square One	Saturday	1083
6	ST-14	Apple Pacific Centre	Monday	3595
7	ST-15	Apple Chinook Centre	Sunday	2926
8	ST-16	Apple Rideau Centre	Monday	2891
9	ST-17	Apple West Edmonton M...	Sunday	2891
10	ST-18	Apple CF Sherway Garde...	Sunday	2912
Total rows: 64		Query complete 00:00:00.668		

Problem statement 11: Identify the least selling product in each country for each year based on total units sold.

Solution:

```
WITH product_ranking
AS
(
SELECT
st.country,
EXTRACT(YEAR FROM sa.sale_date) AS year,
p.product_name,
SUM(sa.quantity) AS quantity,
DENSE_RANK() OVER(PARTITION BY st.country,EXTRACT(YEAR FROM sa.sale_date)
|ORDER BY SUM(sa.quantity) ASC) AS rank
FROM products p
JOIN sales sa
ON p.product_id = sa.product_id
JOIN stores st
ON sa.store_id = st.store_id
GROUP BY st.country, year, p.product_name
)
SELECT * FROM product_ranking
WHERE rank = 1;
```

	country character varying (25) 🔒	year numeric 🔒	product_name character varying (35) 🔒	quantity bigint 🔒	rank bigint 🔒
1	Argentina	2021	iPhone 13 Mini	16	1
2	Argentina	2023	iPhone 15 Pro	99	1
3	Argentina	2023	iPhone 15 Pro Max	99	1
4	Australia	2020	Apple Fitness+	29	1
5	Australia	2021	iPhone 13 Mini	37	1
6	Australia	2022	iPad (10th Gen)	458	1
7	Australia	2023	iPhone 15	264	1
8	Brazil	2020	Apple Fitness+	25	1
9	Brazil	2021	iPhone 13 Mini	30	1
10	Brazil	2022	iPad Pro (M2, 11-inch)	281	1
11	Brazil	2023	iPhone 15	180	1
12	Canada	2019	iPad Pro (10.5-inch)	395	1
Total rows: 131		Query complete 00:00:02.955			

Problem statement 12: Determine the number of warranty claims filed within 180 days of the corresponding product sale.

Solution:

```
SELECT COUNT(*) FROM sales sa
RIGHT JOIN warranty w
    ON sa.sale_id = w.sale_id
WHERE (w.claim_date - sa.sale_date) <= 180;
```

	count bigint 
1	13721

Problem statement 13: For products launched within the last two years, calculate the total units sold and the number of those units with associated warranty claims.

Solution:

```
SELECT
    p.product_id,
    p.product_name,
    COUNT(sa.product_id) AS total_units_sold,
    COUNT(w.claim_id) AS number_of_warranty_claimed
FROM sales sa
LEFT JOIN warranty w
    ON w.sale_id = sa.sale_id
LEFT JOIN products p
    ON sa.product_id = p.product_id
WHERE CURRENT_DATE - p.launch_date <= 730 -- 2 years
GROUP BY p.product_id, p.product_name;
```

	product_id [PK] character varying (10) 	product_name character varying (35) 	total_units_sold bigint 	number_of_warranty_claimed bigint 
1	P-62	iPhone 15	14765	321
2	P-63	iPhone 15 Pro	14857	290
3	P-64	iPhone 15 Pro Max	14737	320

Problem statement 14: Identify the months within the past three years during which sales in the USA exceeded 2500 units.

Solution:

```
SELECT
    TO_CHAR(sa.sale_date, 'MM-YYYY') as month,
    SUM(sa.quantity) AS total_units_sold
FROM sales sa
JOIN stores st
    ON sa.store_id = st.store_id
WHERE st.country = 'USA' AND sa.sale_date >= CURRENT_DATE - INTERVAL '3 years'
GROUP BY month
HAVING SUM(sa.quantity) > 2500;
```

	month 	total_units_sold 
	text	bigint
1	11-2022	3658
2	01-2023	3349
3	10-2022	3662
4	12-2022	3528
5	02-2023	2925
6	03-2023	3417
7	09-2022	2809

Problem statement 15: Identify the product category along with the number of warranty claims filed in the last two years.

Solution:

```
SELECT
    c.category_id,
    c.category_name,
    COUNT(w.claim_id) AS total_warranty_claims
FROM warranty w
JOIN sales sa
    ON w.sale_id = sa.sale_id
JOIN products p
    ON sa.product_id = p.product_id
JOIN category c
    ON p.category_id = c.category_id
WHERE w.claim_date >= CURRENT_DATE - INTERVAL '2 years'
GROUP BY c.category_id, c.category_name
ORDER BY total_warranty_claims DESC;
```

	category_id [PK] character varying (10) 	category_name character varying (20) 	total_warranty_claims bigint 
1	CAT-4	Smartphone	7124
2	CAT-3	Tablet	2231
3	CAT-5	Wearable	947
4	CAT-8	Subscription Service	914
5	CAT-2	Audio	279
6	CAT-10	Accessory	23
7	CAT-1	Laptop	22
8	CAT-6	Streaming Device	1
9	CAT-7	Desktop	1

Problem statement 16: Calculate the percentage of chance of receiving warranty claims after each purchase for each country.

Solution:

```
SELECT
    st.country,
    COUNT(sa.sale_id) AS total_sales,
    COUNT(w.claim_id) AS total_claims,
    ROUND((COUNT(w.claim_id) * 100.0 / COUNT(sa.sale_id)),2) AS warranty_claim_percentage
FROM sales sa
LEFT JOIN warranty w
    ON sa.sale_id = w.sale_id
LEFT JOIN stores st
    ON sa.store_id = st.store_id
GROUP BY st.country
ORDER BY warranty_claim_percentage DESC;
```

	country character varying (25) 🔒	total_sales bigint 🔒	total_claims bigint 🔒	warranty_claim_percentage numeric 🔒
1	UAE	16831	11816	70.20
2	Spain	11848	4414	37.26
3	Italy	12738	1691	13.28
4	Turkey	47320	6157	13.01
5	India	47372	2241	4.73
6	UK	63185	1740	2.75
7	Germany	33732	641	1.90
8	France	25481	335	1.31
9	Netherlands	12874	163	1.27
10	Malaysia	4598	0	0.00
11	Mexico	5876	0	0.00
12	Peru	2889	0	0.00

Problem statement 17: Analyze the year-by-year growth ratio for each store.

Solution:

```
SELECT
  store_id,
  store_name,
  year,
  SUM(final_price) AS total_sales,
  ROUND((((SUM(final_price) - LAG(SUM(final_price)) OVER (PARTITION BY store_id ORDER BY year))*100)
  /LAG(SUM(final_price)) OVER (PARTITION BY store_id ORDER BY year))::NUMERIC,3) AS growth_percentage
FROM
(
SELECT
  st.store_id AS store_id,
  st.store_name AS store_name,
  EXTRACT(YEAR FROM sa.sale_date) AS year,
  (sa.quantity * p.price) AS final_price
FROM stores st
JOIN sales sa
  ON st.store_id = sa.store_id
JOIN products p
  ON sa.product_id = p.product_id
)
GROUP BY store_id, store_name, year;
```

	store_id [PK] character varying (5)	store_name character varying (30)	year numeric	total_sales double precision	growth_percentage numeric
1	ST-1	Apple Fifth Avenue	2019	167255	[null]
2	ST-1	Apple Fifth Avenue	2020	244635	46.265
3	ST-1	Apple Fifth Avenue	2021	1804163	637.492
4	ST-1	Apple Fifth Avenue	2022	3105912	72.153
5	ST-1	Apple Fifth Avenue	2023	1270157	-59.105
6	ST-1	Apple Fifth Avenue	2024	161635	-87.274
7	ST-10	Apple Tysons Corner	2019	161136	[null]
8	ST-10	Apple Tysons Corner	2020	180964	12.305
9	ST-10	Apple Tysons Corner	2021	1689626	833.681
10	ST-10	Apple Tysons Corner	2022	3035130	79.633
11	ST-10	Apple Tysons Corner	2023	1264191	-58.348
12	ST-10	Apple Tysons Corner	2024	159439	-87.388
Total rows: 284		Query complete 00:00:00.567			

Problem statement 18: Calculate the relationship between product price and warranty claims for products sold in the last three years, segmented by price range.

Solution:

```
SELECT
  CASE
    WHEN p.price < 500 THEN 'Affordable(1$-499$)'
    WHEN p.price BETWEEN 500 AND 999 THEN 'Moderate(500$-999$)'
    ELSE 'Premium(999$+)'
  END AS price_segment,
  COUNT(sa.sale_id) AS number_of_total_sales,
  COUNT(w.claim_id) AS number_of_total_claims,
  ROUND((((COUNT(w.claim_id))*100.0)/COUNT(sa.sale_id)),2) AS percentage_of_claims
FROM warranty w
RIGHT JOIN sales sa
  ON w.sale_id = sa.sale_id
LEFT JOIN products p
  ON p.product_id = sa.product_id
WHERE sa.sale_date >= CURRENT_DATE - INTERVAL '3 year'
GROUP BY price_segment;
```

	price_segment text	number_of_total_sales bigint	number_of_total_claims bigint	percentage_of_claims numeric
1	Affordable(1\$-499\$)	91261	5507	6.03
2	Moderate(500\$-999\$)	147258	6502	4.42
3	Premium(999\$+)	61825	1168	1.89

Problem statement 19: Identify the store with the highest percentage of 'Paid Repaired' claims relative to the total number of claims filed.

Solution:

```
SELECT
  st.store_id,
  st.store_name,
  COUNT(w.claim_id) AS total_claims,
  COUNT(CASE WHEN w.repair_status = 'Paid Repaired' THEN 1 END) AS claims_with_paid_repaired,
  ROUND(COALESCE(
    (COUNT(CASE WHEN w.repair_status = 'Paid Repaired' THEN 1 END) * 100.0) / NULLIF(COUNT(w.claim_id),
    0),2) AS percentage_of_paid_repaired
FROM stores st
LEFT JOIN sales sa
  ON st.store_id = sa.store_id
LEFT JOIN warranty w
  ON sa.sale_id = w.sale_id
GROUP BY st.store_id, st.store_name
ORDER BY percentage_of_paid_repaired DESC;
```

	store_id [PK] character varying (5)	store_name character varying (30)	total_claims bigint	claims_with_paid_repaired bigint	percentage_of_paid_repaired numeric
1	ST-26	Apple Kurfürstendamm	160	60	37.50
2	ST-28	Apple Champs-Élysées	151	53	35.10
3	ST-24	Apple Bluewater	259	86	33.20
4	ST-23	Apple Covent Garden	267	86	32.21
5	ST-27	Apple Amsterdam	163	52	31.90
6	ST-22	Apple Regent Street	1214	377	31.05
7	ST-25	Apple Munich	481	147	30.56
8	ST-29	Apple Lyon	184	50	27.17
9	ST-30	Apple Milan	1691	417	24.66
10	ST-31	Apple Dubai Mall	5838	1427	24.44
11	ST-32	Apple Mall of the Emirates	5978	771	12.90
12	ST-43	Apple Jakarta	0	0	0.00
Total rows: 73		Query complete 00:00:00.693			

Problem statement 20: Calculate the 'monthly running total' and the 'rolling average of the last 3 months' of sales for each store.

Solution:

```
SELECT
  st.store_id,
  st.store_name,
  TO_CHAR(DATE_TRUNC('month', sale_date), 'YYYY-MM') AS month,
  SUM(p.price * sa.quantity) as monthly_total_revenue,
  SUM(SUM(p.price * sa.quantity)) OVER
    (PARTITION BY st.store_id ORDER BY TO_CHAR(DATE_TRUNC('month', sa.sale_date), 'YYYY-MM')
     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS monthly_running_total,
  ROUND(AVG(SUM(p.price * sa.quantity)) OVER (
    PARTITION BY st.store_id
    ORDER BY TO_CHAR(DATE_TRUNC('month', sale_date), 'YYYY-MM')
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  )::NUMERIC,2) AS rolling_avg_last_3_months
FROM stores st
LEFT JOIN sales sa
  ON st.store_id = sa.store_id
LEFT JOIN products p
  ON sa.product_id = p.product_id
GROUP BY st.store_id, st.store_name, month;
```

	store_id [PK] character varying (5)	store_name character varying (30)	month text	monthly_total_revenue double precision	monthly_running_total double precision	rolling_avg_last_3_months numeric
1	ST-1	Apple Fifth Avenue	2019-01	4371	4371	4371.00
2	ST-1	Apple Fifth Avenue	2019-02	11760	16131	8065.50
3	ST-1	Apple Fifth Avenue	2019-03	4560	20691	6897.00
4	ST-1	Apple Fifth Avenue	2019-04	2092	22783	6137.33
5	ST-1	Apple Fifth Avenue	2019-05	6138	28921	4263.33
6	ST-1	Apple Fifth Avenue	2019-06	7608	36529	5279.33
7	ST-1	Apple Fifth Avenue	2019-07	7328	43857	7024.67
8	ST-1	Apple Fifth Avenue	2019-08	9225	53082	8053.67
9	ST-1	Apple Fifth Avenue	2019-09	25622	78704	14058.33
10	ST-1	Apple Fifth Avenue	2019-10	34271	112975	23039.33
11	ST-1	Apple Fifth Avenue	2019-11	29815	142790	29902.67
12	ST-1	Apple Fifth Avenue	2019-12	24465	167255	29517.00
Total rows: 3415			Query complete 00:00:03.422			CRLF Ln 20

Problem statement 21: Analyze product sales trends over time, segmented into key periods: from launch to 6 months, 6–12 months, 12–18 months, and beyond 18 months.

Solution:

```
SELECT
  p.product_id,
  p.product_name,
  CASE
    WHEN AGE(sa.sale_date, p.launch_date) < INTERVAL '6 months' THEN '0-6 months'
    WHEN AGE(sa.sale_date, p.launch_date) >= INTERVAL '6 months'
      AND AGE(sa.sale_date, p.launch_date) < INTERVAL '12 months' THEN '6-12 months'
    WHEN AGE(sa.sale_date, p.launch_date) >= INTERVAL '12 months'
      AND AGE(sa.sale_date, p.launch_date) < INTERVAL '18 months' THEN '12-18 months'
    ELSE '18+ months'
  END AS sales_period_from_launch_date,
  SUM(sa.quantity) AS total_units_sold
FROM sales AS sa
JOIN products AS p
  ON sa.product_id = p.product_id
GROUP BY 1, 3
ORDER BY 1 ASC, 4 DESC;
```

	product_id [PK] character varying (10)	product_name character varying (35)	sales_period_from_launch_date text	total_units_sold bigint
1	P-1	MacBook Pro (Touch Bar)	18+ months	2026
2	P-10	Apple TV 4K	18+ months	1707
3	P-10	Apple TV 4K	12-18 months	280
4	P-11	iPhone X	18+ months	18053
5	P-11	iPhone X	12-18 months	690
6	P-12	iPad (6th Gen)	18+ months	15882
7	P-12	iPad (6th Gen)	12-18 months	903
8	P-12	iPad (6th Gen)	6-12 months	307
9	P-13	Apple Watch Series 4	18+ months	10860
10	P-13	Apple Watch Series 4	12-18 months	4552
11	P-13	Apple Watch Series 4	6-12 months	754
12	P-13	Apple Watch Series 4	0-6 months	338
Total rows: 211		Query complete 00:00:00.622		

Conclusion

This project highlights how SQL can drive data-informed decisions in a retail setting by analyzing over half million rows of sales and warranty data. Key insights include identifying top-selling products, tracking seasonal and regional sales trends, evaluating product quality through warranty claims, and assessing customer satisfaction based on service metrics. These findings support strategic improvements in sales, inventory management, product development, and customer service, ultimately boosting profitability and customer loyalty.